NATS SERVER · JETSTREAM · CODE INVESTIGATION

Why source consumers trigger a linear scan

A walk through the stream-sourcing setup paths in `server/stream.go`, the O(S²) behaviour they hide, the data-structure root cause, and what to change.

Repo · **nats-server** | Area · **JetStream sourcing** | Primary file · **server/stream.go** | Date · **2026-06-05**

TL;DR

The per-message data path is **O(1)** — it is *not* the problem. The linear scans live in the **control plane**: consumer setup, leader election, restart and config update. The culprit is `stream.streamSource(iname)` (`stream.go:3793`), an O(S) walk over the `cfg.Sources` slice. Because it is invoked once per source *inside* loops that already iterate over every source, common operations become **O(S²)**. With the 1,000-source scaling scenario that is ~1,000,000 slice steps under the stream lock on every leader change. The root cause is that source *config* lives in a slice with no `iname` index, while runtime state lives in a map.

HOT PATH COST

O(1)

per inbound sourced message — routed via per-source subscription closure

SETUP / ELECTION

O(S²)

`setupSourceConsumers` × `streamSource`

AT S = 1000

~10⁶

slice comparisons per leader election, holding `mset.mu`

Contents

1 What “source consumers” are

A JetStream stream can **source** from other streams. For each entry in its `Sources` list, the sourcing stream creates an internal ephemeral consumer (`JS_SRC_…`) on the origin stream and pulls its messages in.

Each configured source is a `*StreamSource` ; each running source has a matching `*sourceInfo` with the live subscription, sequence tracking and health timer. The two are linked by an **index name** (`iname`) — a composite of stream name, filter and transform (`stream.go:1079` `composeIName()`):

```
// stream.go:1079 — the iname uniquely identifies a source within a stream
func (ssi *StreamSource) composeIName() string {
    // "<name>[:<extHash>][:C=<consumerHash>] <filter/srcs> <dest/transforms>"
    return strings.Join([]string{iName, source, destination}, " ")
}
```

2 The root cause: a slice on one side, a map on the other

The `stream` struct keeps the two halves in different shapes (`stream.go:515` and the config slice):

```
// stream.go:515 — runtime state: keyed by iname, O(1) lookup ✔
sources              map[string]*sourceInfo
sourceSetupSchedules map[string]*time.Timer
```

```
// cfg.Sources — the CONFIG side: a flat slice, no iname index ✘
Sources []*StreamSource
```

> **The mismatch**
> Runtime `sourceInfo` is reachable in O(1) via `mset.sources[iname]`. But the matching `*StreamSource` **config** only lives in the `cfg.Sources` slice — there is no `map[iname]*StreamSource`. Any time code holds an `iname` and needs the config (external API prefix, filter, transforms, start policy), it must walk the slice.

3 The scan itself

That walk is one tiny, innocuous-looking function:

```
// stream.go:3793
func (mset *stream) streamSource(iname string) *StreamSource {
    for _, ssi := range mset.cfg.Sources {   // O(S) linear walk

        if ssi.iname == iname {

            return ssi
        }
    }
    return nil
}
```

On its own this is cheap. The cost comes entirely from *where* it is called — almost always from inside another loop that is itself iterating over every source.

4 Where it is invoked, and why

None of these are on the message data path; all are control-plane operations that fire on **startup, leader election, restart, health-driven re-setup, config update** and `STREAM.INFO`.

4a · `setupSourceConsumers` → O(S²) control

Runs on every leader election / R1 startup. It loops over all sources; each iteration ends up calling `streamSource(iname)` deep inside `trySetupSourceConsumer`:

```
// stream.go:4805
func (mset *stream) setupSourceConsumers() error {
    ...
    mset.startingSequenceForSources()                // reverse store scan (4b)
    for _, ssi := range mset.cfg.Sources {           // S iterations

        if si := mset.sources[ssi.iname]; si != nil {
            mset.setupSourceConsumer(ssi.iname, si.sseq+1, ...)  // → trySetupSou

        }
    }
}

// stream.go:3911 — inside trySetupSourceConsumer, per source:
ssi := mset.streamSource(iname)   // O(S) walk × S sources = O(S²)
```

4b · `startingSequenceForSources` → reverse store scan + inner O(S)

control

To resume each source at the right sequence, this reads the stream's own messages backwards looking for `Nats-Stream-Source` headers. For legacy (pre-2.10) headers that carry only a stream name and no `iname`, it must match by scanning every configured source:

```
// stream.go:4787
streamName, iName, sseq := streamAndSeq(bytesToString(ss))
if iName == _EMPTY_ {  // pre-2.10 header → must search all sources
    for _, ssi := range mset.cfg.Sources {   // O(S) per scanned message

        if streamName == ssi.Name || (ssi.External != nil && ...) {
            update(ssi.iname, sseq)
        }
    }
}
```

4c · `setStartingSequenceForSources` → the explicitly-flagged O(S²)

config update

The config-update sibling of 4b. Here the inner loop calls `streamSource()` **twice** per iteration — the only place in the tree with a standing `TODO` about the walk:

```
// stream.go:4638
} else if indexName == _EMPTY_ && streamName != _EMPTY_ {
```

```
    for iName := range iNames {
        // TODO streamSource is a linear walk, to optimize later

        if si := mset.sources[iName]; si != nil && streamName == si.name ||

            (mset.streamSource(iName).External != nil &&       // walk #1

             streamName == si.name+":"+getHash(mset.streamSource(iName).External.

            ...
        }
    }
}
```

Bonus O(S²) in the same update path

The update routine also derives a per-source consumer name via `getSourcingConsumerIName` (`stream.go:2509`), which calls `createSourcingConsumerHash` (`consumer.go:6354`) — and *that* loops over all sources to detect duplicate stream names. Called once per source in two separate loops (`stream.go:2516, 2523`), it is another independent O(S²) factor.

4d · `sourceInfo` → O(S) per source on `STREAM.INFO` monitoring

```
// stream.go:2992 — building StreamSourceInfo for the API response
} else if ss := mset.streamSource(si.iname); ss != nil && ss.External != nil {
```

`sourcesInfo()` (`stream.go:2953`) calls this for every source, so a single `$JS.API.STREAM.INFO` on a 1,000-source stream is itself O(S²).

5 The hot path is clean — by design

It is worth stating clearly: **inbound message processing does not scan.** When a source consumer is set up, the delivery subscription captures its own `si` in a closure, and queues messages already tagged with it:

```
// stream.go:4161 — each source's own sub closes over its own si
sub, _ := mset.subscribeInternal(deliverSubject, func(...) {
    mset.queueInbound(msgs, subject, reply, hdr, msg,
 si
```

```
, nil)  // si is bound, no lookup
})

// stream.go:4314 — receives the si directly, O(1)
func (mset *stream) processInboundSourceMsg(
si *sourceInfo
, m *inMsg) bool {
```

All sources are also multiplexed onto a single goroutine (`processAllSourceMsgs` , `stream.go:4187`) to avoid lock contention and thread thrashing. So the scaling pain is strictly about *control-plane* events, not throughput.

6 Complexity at a glance

● O(1) / fine ● O(S) linear ● O(S²) quadratic

S = number of configured sources, M = messages scanned backwards

OPERATION	TRIGGER	LOCATION	COST
Inbound sourced message	per message	stream.go:4314	O(1)
`streamSource(iname)`	helper	stream.go:3793	O(S)
`setupSourceConsumers`	leader election / R1 start	stream.go:4805	O(S²)
`startingSequenceForSources`	setup / restart	stream.go:4694	O(M·S) → O(S²)*
`setStartingSequenceForSources`	config update	stream.go:4566	O(M·S²)*
Config update (sources)	`STREAM.UPDATE`	stream.go:2504	O(S²)
`sourcesInfo` / `sourceInfo`	`STREAM.INFO`	stream.go:2953 / 2992	O(S²)
Health re-setup of stalled sources	periodic ticker	stream.go:4232	O(stalled·S)

* The O(S²) factors apply when origin messages carry legacy (pre-2.10) source headers without an `iname` ; modern headers short-circuit the inner walk via the `iNames` map but the outer `streamSource` -driven O(S²) in setup still applies.

7 Why it matters at scale

The repo ships a deliberately-skipped scaling test that frames the intended workload:

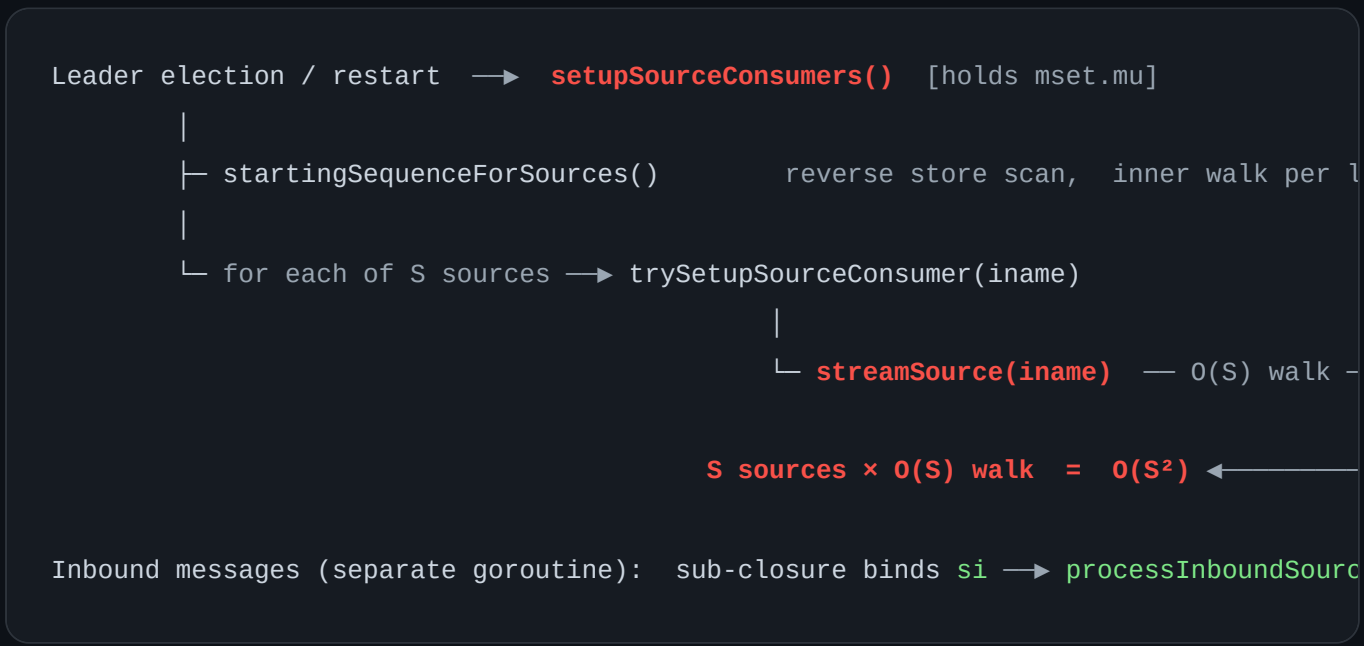
```go
// jetstream_sourcing_scaling_test.go:110
func TestStreamSourcingScalingSourcingManyBenchmark(t *testing.T) {
    t.Skip()
    var numSourced     = 1000      // one sourcing stream pulling from 1000 strea
    var numMsgPerSource = 10_000
```

WHERE THE TIME GOES

Every leader election, server restart, or scale-up of the sourcing stream runs `setupSourceConsumers`. At S=1000 that is ~10^6 `iname` string comparisons walking the slice — all while holding `mset.mu`, blocking publishes, info requests and other consumer setup.

IT COMPOUNDS

Health checks re-setup stalled sources (each O(S)); a flurry of stalls after a network blip multiplies the walks. Monitoring tools polling `STREAM.INFO` add their own $O(S^2)$ on top, under the same lock.



8 The fix

Give the config side the same O(1) index the runtime side already has.

> **Core change**
>
> Maintain a `map[string]*StreamSource` keyed by `iname` next to `cfg.Sources`, rebuilt wherever the sources list is (re)constructed — `resetSourceInfo` (`stream.go:4659`) and the update path (`stream.go:2523`). Then `streamSource()` becomes a single map read.

```
// streamSource() after: O(1)
func (mset *stream) streamSource(iname string) *StreamSource {

    return mset.cfgSources[iname]   // map lookup, populated alongside cfg.Source

}
```

Knock-on simplifications once the index exists:

- **`setupSourceConsumers`** drops from O(S²) to **O(S)** with no other change.
- The two inner `streamSource()` calls in `setStartingSequenceForSources` (`stream.go:4641-4642`) — the standing `TODO` — collapse to map reads.
- `sourceInfo` / `sourcesInfo` on `STREAM.INFO` drop from O(S²) to **O(S)**.
- For the legacy-header matching (4b/4c) and the duplicate detection in `createSourcingConsumerHash` (`consumer.go:6354`), a secondary `map[streamName][]*StreamSource` removes the remaining by-name walks.

> **Why it's safe**
>
> `iname` is already validated to be unique per stream (`stream.go:1963-1974`, “check sources for duplicates”), so a map keyed by it is well-defined. The map is pure derived state under `mset.mu`; it changes performance characteristics only, not behaviour. The existing skipped scaling test is the natural acceptance gate.

Key references

SYMBOL	FILE:LINE	ROLE
`streamSource`	stream.go:3793	the linear scan

SYMBOL	FILE:LINE	ROLE
`sources` map / `cfg.Sources` slice	stream.go:515	the asymmetry
`composeIName`	stream.go:1079	iname construction
`setupSourceConsumers`	stream.go:4805	O(S²) setup loop
`startingSequenceForSources`	stream.go:4694	reverse-scan resume
`setStartingSequenceForSources`	stream.go:4566	TODO-flagged O(S²)
`createSourcingConsumerHash`	consumer.go:6354	by-name dedup walk
`processInboundSourceMsg`	stream.go:4314	O(1) hot path
`TestStreamSourcingScaling…`	jetstream_sourcing_scaling_test.go:110	1000-source workload

Generated for the `claude/source-consumers-linear-scan` investigation · NATS Server JetStream. All line numbers refer to the working tree at the time of writing. This report describes existing behaviour and a proposed direction; no source files were modified.